

Adaptive Recommendation System: User Segmentation, Anomaly Detection, Personalized Ranking, and Reinforcement Learning

Python | NumPy | TensorFlow | From-Scratch Optimization

Timur Filimonov

Project Overview

This project focuses on core machine learning methods beyond supervised regression and classification, covering unsupervised learning, recommender systems, and reinforcement learning. The work includes implementing K-means clustering for grouping similar users, Gaussian anomaly detection for identifying unusual patterns, collaborative and neural recommendation models for personalized ranking, and Deep Q-Networks for sequential decision-making.

The project was built to strengthen understanding of both the mathematical logic and practical implementation of modern machine learning systems that learn from structure, interaction, and feedback rather than direct labels alone.

Imports and Setup

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Dot
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import MSE

np.random.seed(42)
tf.random.set_seed(42)
```

Dataset / Input Construction

This project uses synthetic user, item, and interaction data to demonstrate clustering, anomaly detection, recommendation, and reinforcement learning in one integrated system.

```
num_users = 120
num_movies = 80
num_user_features = 6
num_item_features = 6
num_outputs = 32
k_clusters = 4
lambda_ = 0.01
gamma = 0.99
ALPHA = 1e-3

user_features = np.random.rand(num_users, num_user_features)
user_features[:, 0] = 2.5 + 2.5 * user_features[:, 0]
user_features[:, 4] = 5 + 45 * user_features[:, 4]
user_features[:, 5] = 10 + 190 * user_features[:, 5]

item_features = np.random.rand(num_movies, num_item_features)
item_features[:, 4] = 10 + 90 * item_features[:, 4]
item_features[:, 5] = 2.0 + 3.0 * item_features[:, 5]

Y = np.zeros((num_movies, num_users))
R = np.zeros((num_movies, num_users), dtype=int)

for i in range(num_movies):
    for j in range(num_users):
        if np.random.rand() < 0.35:
            R[i, j] = 1
            user_taste = np.mean(user_features[j, 1:4])
            item_style = np.mean(item_features[i, 0:3])

            rating = (
                2.5
                + 2.0 * (0.6 * user_taste + 0.4 * item_style)
                + np.random.normal(0, 0.3)
            )
            rating = np.clip(rating, 1.0, 5.0)
            Y[i, j] = rating
```

The synthetic ratings are generated from a weighted combination of user preference patterns, item characteristics, and random noise, then clipped to the standard 1–5 rating range.

```

print("User feature matrix shape:", user_features.shape)
print("Item feature matrix shape:", item_features.shape)
print("Ratings matrix shape:", Y.shape)
print("Rating indicator matrix shape:", R.shape)
print("Observed ratings:", np.sum(R))

```

The generated user features represent rating behavior, preference intensity, and interaction activity. The item features represent content characteristics and popularity. From these, a user-item ratings matrix and rating indicator matrix were constructed to support all later stages of the project.

K-Means Clustering

Objective

The first part of the project applies K-means clustering to group users with similar behavior patterns. This helps identify segments such as high-engagement users, users with stronger preferences for certain content styles, or more casual users.

Functions Implemented:

- find_closest_centroids
- compute_centroids
- K-means iteration loop

Code:

```

def find_closest_centroids(X, centroids):
    K = centroids.shape[0]
    idx = np.zeros(X.shape[0], dtype=int)

    for i in range(X.shape[0]):
        min_dist = float('inf')

        for j in range(K):
            dist = np.sum((X[i] - centroids[j]) ** 2)

            if dist < min_dist:
                min_dist = dist
                idx[i] = j

    return idx

def compute_centroids(X, idx, K):
    m, n = X.shape
    centroids = np.zeros((K, n))

```

```

    for k in range(K):
        points = X[idx == k]

        if len(points) > 0:
            centroids[k] = np.mean(points, axis=0)
        else:
            centroids[k] = X[np.random.randint(0, X.shape[0])]

    return centroids

def run_kmeans(X, initial_centroids, num_iters=10):
    centroids = initial_centroids

    for _ in range(num_iters):
        idx = find_closest_centroids(X, centroids)
        centroids = compute_centroids(X, idx, centroids.shape[0])

    return centroids, idx

random_indices = np.random.choice(user_features.shape[0], k_clusters, replace=False)
initial_centroids = user_features[random_indices]
centroids, user_clusters = run_kmeans(user_features, initial_centroids, num_iters=10)

print("Final centroids:\n", centroids)
print("Cluster assignments:\n", user_clusters[:20])
print("Cluster counts:", np.bincount(user_clusters))

cluster_names = {
    0: "High-Engagement Users",
    1: "Selective Viewers",
    2: "Broad-Interest Users",
    3: "Low-Activity Users"
}

```

```
for cluster_id, name in cluster_names.items():
    print(f'Cluster {cluster_id}: {name}')
```

```
plt.figure(figsize=(6, 4))
```

```
plt.scatter(
    user_features[:, 0],
    user_features[:, 4],
    c=user_clusters,
    cmap='viridis',
    alpha=0.7
)
```

```
plt.xlabel("Average Rating")
```

```
plt.ylabel("Watch Frequency")
```

```
plt.title("User Clusters from K-Means")
```

```
plt.show()
```

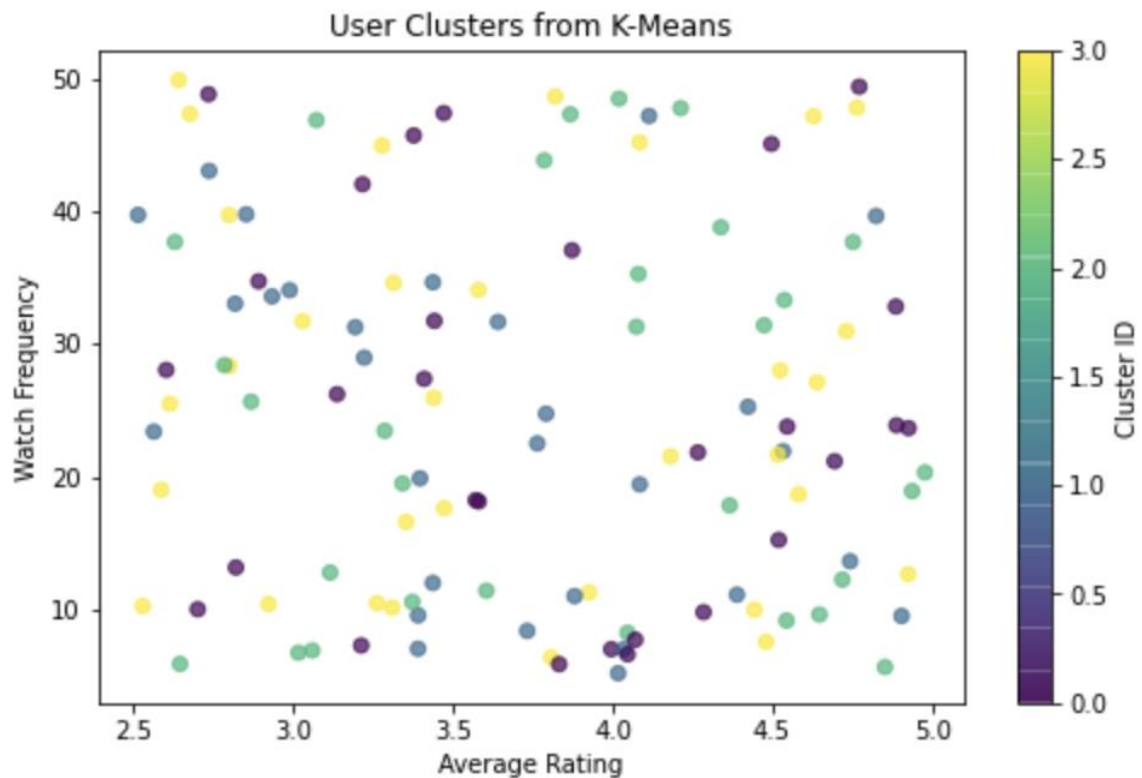
Cluster 0: High-Engagement Users

Cluster 1: Selective Viewers

Cluster 2: Broad-Interest Users

Cluster 3: Low-Activity Users

Cluster counts: [29 29 30 32]



Interpretation of the Code:

K-means was used to discover user groups directly from feature similarity. Cluster counts, labels, and a scatter plot were added to make the discovered user segments easier to interpret.

Gaussian Anomaly Detection

Objective

The second part of the project applies anomaly detection to identify unusual user or item behavior. This may represent noisy data, suspicious patterns, or rare cases that should be analyzed separately.

Functions Implemented

- estimate_gaussian
- select_threshold

Code:

```
def estimate_gaussian(X):
    mu = np.mean(X, axis=0)
    var = np.var(X, axis=0)
    return mu, var

def multivariate_gaussian(X, mu, var):
    k = len(mu)
    var = var + 1e-6
    var_matrix = np.diag(var)
    X = X - mu

    coeff = 1 / np.sqrt((2 * np.pi) ** k * np.linalg.det(var_matrix))
    exp_term = np.exp(-0.5 * np.sum((X @ np.linalg.inv(var_matrix)) * X, axis=1))

    return coeff * exp_term
```

```

def select_threshold(y_val, p_val):
    best_epsilon = 0
    best_F1 = 0
    F1 = 0

    step_size = (max(p_val) - min(p_val)) / 1000

    for epsilon in np.arange(min(p_val), max(p_val), step_size):
        predictions = (p_val < epsilon)

        tp = np.sum((predictions == 1) & (y_val == 1))
        fp = np.sum((predictions == 1) & (y_val == 0))
        fn = np.sum((predictions == 0) & (y_val == 1))

        prec = tp / (tp + fp) if (tp + fp) > 0 else 0
        rec = tp / (tp + fn) if (tp + fn) > 0 else 0

        F1 = (2 * prec * rec) / (prec + rec) if (prec + rec) > 0 else 0

        if F1 > best_F1:
            best_F1 = F1
            best_epsilon = epsilon

    return best_epsilon, best_F1

anomaly_input = user_features[:, [0, 4, 5]]
mu, var = estimate_gaussian(anomaly_input)
p_vals = multivariate_gaussian(anomaly_input, mu, var)

print("Mean:", mu)
print("Variance:", var)
print("Probability values:", p_vals[:10])

epsilon = np.percentile(p_vals, 5)
anomalies = p_vals < epsilon

print("Chosen threshold:", epsilon)
print("Detected anomalies:", np.sum(anomalies))
print("Sample anomaly indices:", np.where(anomalies)[0][:10])

```

Interpretation of the Code:

Gaussian anomaly detection was applied to user behaviour features. Users with very low probability values were flagged as anomalies, representing unusual or rare interaction patterns.

Collaborative Filtering Cost Function implementation

Objective

This section focuses on implementing the collaborative filtering objective and regularization term. It verifies the mathematical structure used to measure recommendation error, rather than fully training a latent factor recommendation model.

Functions Implemented:

- `cofi_cost_func`
- `sq_dist`

Code:

```
def sq_dist(a, b):
    d = np.sum((a - b) ** 2)
    return d

def cofi_cost_func(X, W, b, Y, R, lambda_):
    nm, nu = Y.shape
    J = 0

    for i in range(nm):
        for j in range(nu):
            if R[i, j] == 1:
                prediction = np.dot(W[j], X[i]) + b[0, j]
                error = prediction - Y[i, j]
                J += error ** 2

    J = J / 2
    J += (lambda_ / 2) * (np.sum(W ** 2) + np.sum(X ** 2))

    return J
```



```

num_features_cf = 5
X_cf = np.random.rand(num_movies, num_features_cf)
W_cf = np.random.rand(num_users, num_features_cf)
b_cf = np.random.rand(1, num_users)

cost = cofi_cost_func(X_cf, W_cf, b_cf, Y, R, lambda_)
print("Collaborative filtering cost:", cost)

```

Interpretation of the Code:

This part shows how recommendation models can learn latent relationships between users and items by minimizing rating prediction error. Regularization was added to reduce overfitting and improve generalization.

Neural Recommendation Architecture

Objective

A two-tower neural recommendation architecture was built to learn compact user and item embeddings. These embeddings are normalized and compared through a dot product to generate recommendation scores.

Functions Implemented:

- user neural network
- item neural network
- embedding comparison through dot product

Code:

```

user_NN = tf.keras.models.Sequential([
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(num_outputs, activation='linear')
])

item_NN = tf.keras.models.Sequential([
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(num_outputs, activation='linear')
])

```

```

input_user = tf.keras.layers.Input(shape=(num_user_features,))
vu = user_NN(input_user)
vu = tf.linalg.l2_normalize(vu, axis=1)

input_item = tf.keras.layers.Input(shape=(num_item_features,))
vm = item_NN(input_item)
vm = tf.linalg.l2_normalize(vm, axis=1)

output = tf.keras.layers.Dot(axes=1)([vu, vm])

model = tf.keras.Model([input_user, input_item], output)
model.summary()

sample_user = user_features[:10].astype(np.float32)
sample_item = item_features[:10].astype(np.float32)

target_scores = np.random.rand(len(sample_user), 1).astype(np.float32)

model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
model.fit([sample_user, sample_item], target_scores, epochs=10, verbose=0)

scores = model.predict([sample_user, sample_item])
print("Predicted scores after training:\n", scores)

```

Interpretation of the Code:

The neural recommender learns dense representations for both users and items. This allows the system to move beyond direct rating matching and instead compare learned embeddings that capture deeper structure in user preferences and item characteristics.

Deep Q-Networks Components

Objective

The final part of the project introduces reinforcement learning. A Deep Q-Network was built to model sequential decision-making, where the system learns to choose actions that maximize long-term reward.

Functions Implemented:

- Q-network
- target Q-network
- DQN loss function

Code:

```
state_size = (8, )
num_actions = 4

q_network = Sequential([
    Input(shape=state_size),
    Dense(64, activation='relu'),
    Dense(64, activation='relu'),
    Dense(num_actions, activation='linear')
])

target_q_network = Sequential([
    Input(shape=state_size),
    Dense(64, activation='relu'),
    Dense(64, activation='relu'),
    Dense(num_actions, activation='linear')
])

optimizer = Adam(learning_rate=ALPHA)

def compute_loss(experiences, gamma, q_network, target_q_network):
    states, actions, rewards, next_states, done_vals = experiences

    max_qsa = tf.reduce_max(target_q_network(next_states), axis=-1)
    y_targets = rewards + (1 - done_vals) * gamma * max_qsa

    q_values = q_network(states)
    q_values = tf.gather_nd(
        q_values,
        tf.stack([tf.range(q_values.shape[0]), tf.cast(actions, tf.int32)], axis=1)
    )

    loss = MSE(y_targets, q_values)
    return loss

batch_size = 8
states = tf.random.normal((batch_size, 8))
actions = tf.random.uniform((batch_size,), minval=0, maxval=num_actions, dtype=tf.int32)
rewards = tf.random.uniform((batch_size,), minval=0, maxval=5, dtype=tf.float32)
next_states = tf.random.normal((batch_size, 8))
done_vals = tf.cast(tf.random.uniform((batch_size,), minval=0, maxval=2, dtype=tf.int32),
tf.float32)
```

```
experiences = (states, actions, rewards, next_states, done_vals)
loss = compute_loss(experiences, gamma, q_network, target_q_network)

print("DQN loss:", loss.numpy())
```

Interpretation of the Code:

The DQN component demonstrates how reinforcement learning can extend recommendation systems beyond one-step prediction. Instead of only estimating immediate relevance, the model can learn strategies that improve longer-term outcomes through repeated interactions.

Integrated System Design

The full system can be viewed as a unified recommendation pipeline. First, synthetic user and item data are constructed. K-means clustering segments users into behavioural groups. Gaussian anomaly detection then flags unusual profiles that may represent noisy or rare patterns. The collaborative filtering objective provides a mathematical error structure for recommendation, while the neural two-tower model learns user and item embeddings for score prediction. Finally, the DQN components provide a foundation for future long-term recommendation strategy optimisation.

Results and Key Takeaways

This project combined multiple machine learning paradigms into one coherent recommendation pipeline. K-means clustering helped identify user segments, anomaly detection highlighted unusual behaviours, collaborative and neural recommendation methods supported personalization, and reinforcement learning introduced sequential optimization.

The project strengthened my practical understanding of:

- unsupervised pattern discovery
- probability-based anomaly detection
- latent factor recommendation methods
- neural embedding architectures
- action-value learning with Deep Q-Networks

Skills Demonstrated:

- Unsupervised Learning
- K-Means Clustering
- Gaussian Anomaly Detection
- Collaborative Filtering
- Neural Recommenders
- TensorFlow / Keras
- Reinforcement Learning
- Deep Q-Networks
- NumPy
- Model Design
- Representation Learning
- Sequential Decision-Making